Lab 4: Microservices and Cloud-Native Design - Shuruq Adel (SE2)

Introduction

The objective of this lab was to design and deploy a small **cloud-native microservices system** using Docker and Docker Compose.

By completing this lab, the following goals were achieved:

- Implement two independent microservices using Python Flask
- Containerize each service using Docker
- Enable service-to-service communication
- Apply cloud-native and 12-factor application principles
- Observe failure propagation and resilience behavior

---

1. Implementation Summary

Product Service

- Stores products in memory
- Provides:
  - /health endpoint
  - /products/<id> endpoint
- Stateless design (no local storage)

Example product data:

- Laptop — 1200
- Phone — 650
- Headphones — 120

Order Service

- Accepts order requests
- Calls product-service using HTTP
- Uses **environment variables** for configuration
- Implements retry and timeout logic

Key cloud-native concepts used:

- Externalized configuration
- Stateless services
- Service discovery via Docker networking

---

2. Containerization

Each service was packaged independently.

Docker Concepts Observed

- Separate Dockerfile per microservice
- Independent deployment units
- Lightweight Python slim images
- Port exposure for API communication

Docker Compose was used to:

- Build containers
- Create network automatically
- Manage service startup order
- Apply health checks and restart policies

---

3. Running the System

```
PS C:\Users\shuruq.DESKTOP-AR7JK5Q> docker compose up --build -d
[+] Building 15.2s (14/14) FINISHED
 => product-service built
 => order-service built
[+] Running 3/3
 ✓ Network week4-lab_default     Created
 ✓ Container product-service     Started
 ✓ Container order-service       Started


PS C:\Users\shuruq.DESKTOP-AR7JK5Q> docker compose ps
NAME              IMAGE                    COMMAND          SERVICE          STATUS
PORTS
product-service   week4-lab-product-service "python app.py"   product-service
running (healthy)   0.0.0.0:5001->5001/tcp
order-service     week4-lab-order-service   "python app.py"   order-service
running (healthy)   0.0.0.0:5002->5002/tcp
```

---

4. Functional Testing

Testing product-service

```
PS C:\Users\shuruq.DESKTOP-AR7JK5Q> curl http://localhost:5001/health
```

{"service":"product-service","status":"up"}

```
PS C:\Users\shuruq.DESKTOP-AR7JK5Q> curl http://localhost:5001/products/1
{"id":1,"name":"Laptop","price":1200}
```

```
PS C:\Users\shuruq.DESKTOP-AR7JK5Q> curl http://localhost:5001/products/99
{"error":"Product not found"}
```

Testing order-service

```
PS C:\Users\shuruq.DESKTOP-AR7JK5Q> curl -X POST http://localhost:5002/orders
-H "Content-Type: application/json" -d "{\"product_id\":1,\"quantity\":2}"
{"message":"Order created","product":"Laptop","quantity":2,"total_price":2400}
```

✅ Order created successfully using data retrieved from product-service.

---

5. **Failure Testing (Resilience Observation)**

A service failure was simulated by stopping **product-service**.

```
PS C:\Users\shuruq.DESKTOP-AR7JK5Q> docker stop product-service
product-service

PS C:\Users\shuruq.DESKTOP-AR7JK5Q> curl -X POST http://localhost:5002/orders
-H "Content-Type: application/json" -d "{\"product_id\":2,\"quantity\":2}"
{"error":"product-service unavailable"}
```

Observation

- order-service attempted retries
- Request failed with **503 Service Unavailable**
- Demonstrated dependency between services

Service Recovery

```
PS C:\Users\shuruq.DESKTOP-AR7JK5Q> docker start product-service
product-service
```

```
PS C:\Users\shuruq.DESKTOP-AR7JK5Q> docker compose ps
NAME            IMAGE                    COMMAND          SERVICE          STATUS
product-service   week4-lab-product-service running (healthy)
order-service     week4-lab-order-service   running (healthy)

PS C:\Users\shuruq.DESKTOP-AR7JK5Q> curl -X POST http://localhost:5002/orders
-H "Content-Type: application/json" -d "{\"product_id\":1,\"quantity\":2}"
{"message":"Order created","product":"Laptop","quantity":2,"total_price":2400}
```

🟩 System recovered after service restart.

---

6. **Cloud-Native Principles Demonstrated**

This implementation followed multiple cloud-native practices:

- Stateless services
- Containerization
- Environment-based configuration
- Health monitoring endpoints
- Restart policies
- Service discovery using container names

---

7. **Benefits of Microservices**

- Independent deployment of services
- Clear separation of responsibilities
- Easier scalability per service
- Technology flexibility

---

8. **Challenges Observed**

- Increased configuration complexity
- Network dependency between services
- Failure propagation across services
- More operational management required

---

9. Architecture Reflection

1. Benefits compared to monolith

Separating product and order logic into different services allows independent development, deployment, and scaling.

---

2. New complexities introduced

Microservices require networking, environment configuration, and container orchestration which increases operational complexity.

---

3. Effect of latency or slow services

If product-service becomes slow or unavailable, order-service also slows or fails because communication is synchronous.

---

4. 12-Factor Principles Observed

- Configuration via environment variables
- Stateless processes
- Port binding
- Disposable containers

- Logs handled by Docker

---

10. Learning Outcomes

After completing this lab, I learned how to:

- Design microservices architecture
- Containerize applications with Docker
- Use Docker Compose orchestration
- Implement service communication
- Test resilience and failure behavior

Conclusion

This lab demonstrated how cloud-native microservices operate in practice. Docker and Docker Compose simplified deployment while highlighting real-world distributed system challenges such as service dependency, resilience, and operational complexity.

The experiment showed that while microservices provide flexibility and scalability, they also introduce networking and coordination challenges that must be managed carefully.